



Red Hat OpenShift AI Self-Managed 3.0

Customize models to build gen AI applications

Red Hat OpenShift AI Self-Managed 3.0 Customize models to build gen AI applications

Legal Notice

Copyright © 2025 Red Hat, Inc.

The text of and illustrations in this document are licensed by Red Hat under a Creative Commons Attribution–Share Alike 3.0 Unported license ("CC-BY-SA"). An explanation of CC-BY-SA is available at

<http://creativecommons.org/licenses/by-sa/3.0/>

. In accordance with CC-BY-SA, if you distribute this document or an adaptation of it, you must provide the URL for the original version.

Red Hat, as the licensor of this document, waives the right to enforce, and agrees not to assert, Section 4d of CC-BY-SA to the fullest extent permitted by applicable law.

Red Hat, Red Hat Enterprise Linux, the Shadowman logo, the Red Hat logo, JBoss, OpenShift, Fedora, the Infinity logo, and RHCE are trademarks of Red Hat, Inc., registered in the United States and other countries.

Linux[®] is the registered trademark of Linus Torvalds in the United States and other countries.

Java[®] is a registered trademark of Oracle and/or its affiliates.

XFS[®] is a trademark of Silicon Graphics International Corp. or its subsidiaries in the United States and/or other countries.

MySQL[®] is a registered trademark of MySQL AB in the United States, the European Union and other countries.

Node.js[®] is an official trademark of Joyent. Red Hat is not formally related to or endorsed by the official Joyent Node.js open source or commercial project.

The OpenStack[®] Word Mark and OpenStack logo are either registered trademarks/service marks or trademarks/service marks of the OpenStack Foundation, in the United States and other countries and are used with the OpenStack Foundation's permission. We are not affiliated with, endorsed or sponsored by the OpenStack Foundation, or the OpenStack community.

All other trademarks are the property of their respective owners.

Abstract

Learn how to customize a model, from setting up your development environment to building and deploying a model specific to your domain-specific use case.

Table of Contents

PREFACE	3
CHAPTER 1. OVERVIEW OF THE MODEL CUSTOMIZATION WORKFLOW	4
CHAPTER 2. SET UP YOUR WORKING ENVIRONMENT	5
2.1. ABOUT THE RED HAT PYTHON INDEX	5
2.2. MIRROR THE PYTHON INDEX FOR YOUR DISCONNECTED ENVIRONMENT	6
2.3. INSTALL PACKAGES	6
2.4. IMPORT EXAMPLE NOTEBOOKS	7
2.4.1. Clone an example Git repository	7
CHAPTER 3. PREPARE YOUR DATA FOR AI CONSUMPTION	9
3.1. PROCESS DATA BY USING DOCLING	9
3.2. EXPLORE THE DATA PROCESSING EXAMPLES	9
3.3. AUTOMATE DATA PROCESSING STEPS BY BUILDING AI PIPELINES	10
3.4. EXPLORE THE KUBEFLOW PIPELINE EXAMPLES	10
CHAPTER 4. GENERATE SYNTHETIC DATA	11
4.1. EXPLORE THE SDG HUB EXAMPLES	11
4.2. GUIDED EXAMPLE - BUILD A KFP PIPELINE FOR SDG	12
CHAPTER 5. TRAIN THE MODEL BY USING YOUR PREPARED DATA	13
5.1. EXPLORE THE TRAINING HUB EXAMPLES	13
5.2. ESTIMATE MEMORY USAGE	13
5.3. COMPARE THE PERFORMANCE OF OSFT AND SFT TRAINING ALGORITHMS	14
5.4. DISTRIBUTE TRAINING JOBS BY USING THE KUBEFLOW TRAINER OPERATOR	14
5.5. DISTRIBUTE FINE-TUNING WITH TRAINING HUB AND KUBEFLOW TRAINER	15
CHAPTER 6. END-TO-END MODEL CUSTOMIZATION WORKFLOW	16
CHAPTER 7. SUPPORT PHILOSOPHY: A SECURE PLATFORM	17

PREFACE

Learn how to customize a model, from setting up your development environment to building and deploying a model specific to your domain-specific use case.

CHAPTER 1. OVERVIEW OF THE MODEL CUSTOMIZATION WORKFLOW

Red Hat AI model customization empowers you to tailor artificial intelligence models to your unique data and operational requirements. The model customization process involves the training or fine-tuning of pre-existing models with proprietary datasets, followed by their deployment with specific configurations on the Red Hat OpenShift AI platform. This comprehensive approach is facilitated by a powerful suite of integrated toolkits that streamline and accelerate the development of generative AI applications.

The workflow for customizing models includes the following tasks:

Set up your working environment

Ensure reliable and secure access to supported libraries with the Red Hat Hosted Python index. For details, see [Set up your working environment](#).

Prepare your data for AI consumption

To prepare your data, use Docling, a powerful Python library to transform unstructured data (such as text documents, images, and audio files) into structured formats that models can consume. For details, see [Prepare your data for AI consumption](#).

To automate data processing tasks, you can build Kubeflow Pipelines (KFP), see [Automate data processing steps by building AI pipelines](#).

Generate synthetic data

Use the Red Hat AI Synthetic Data Generation (SDG) Hub framework to build, compose, and scale synthetic data pipelines with modular blocks. With the SDG Hub, you can extend your synthetic data pipelines with custom blocks to fit your domain, replace ad hoc scripts with the SDG Hub repeatable framework, and scale data generation with asynchronous execution and monitoring. For details, see [Generate synthetic data](#).

Train a model by using your prepared data

After you prepare your data, use the Red Hat AI Training Hub to simplify and accelerate the process of fine-tuning and customizing a foundation model by using your own data.

You can extend a base notebook to use distributed training across multiple nodes by using the KubeFlow Trainer Operator (KFTO). The KFTO, abstracts the underlying infrastructure complexity of distributed training and fine-tuning of models. The iterative process of fine-tuning significantly reduces the time and resources required compared to training models from scratch.

For details, see [Train a model by using your prepared data](#).

Serve and consume a customized model

After you customize a model, you can serve your customized models as APIs (Application Programming Interfaces). Serving a model as an API enables seamless integration into existing or newly developed applications.

Learn more about serving and consuming a customized model [Deploying models on the model serving platform](#).

CHAPTER 2. SET UP YOUR WORKING ENVIRONMENT

To set up your working environment for customizing models, complete these tasks:

1. For disconnected environments, mirror the Python index.
2. Create a custom workbench image that is based on a base image that is configured to use the Red Hat Python index and install packages.
3. From your running workbench, import example notebooks.

2.1. ABOUT THE RED HAT PYTHON INDEX

Red Hat AI includes a maintained Python package index that provides secure and reliable access to supported libraries, with full support for disconnected environments. For details about Red Hat support for the Python package index, see [Support philosophy: A secure platform](#).

Table 2.1 lists the images that are configured to use the Red Hat Python index.

Table 2.1. Images configured to use the Red Hat Python index

Accelerator UBI9	List of packages	Registry URL	Catalog URL
CPU	https://console.redhat.com/api/pypi/public-rhai/rhoai/3.0/cpu-ubi9/simple/	registry.redhat.io/rhai/base-image-cpu-rhel9:3.0.0-1763044919	https://catalog.redhat.com/software/containers/rhai/base-image-cpu-rhel9/690377f9d1c73dd1e81192f0
CUDA	https://console.redhat.com/api/pypi/public-rhai/rhoai/3.0/cuda-ubi9/simple/	registry.redhat.io/rhai/base-image-cuda-rhel9:3.0.0-1763044928	https://catalog.redhat.com/software/containers/rhai/base-image-cuda-rhel9/690377f9e1522d6afa972cc6
ROCm	https://console.redhat.com/api/pypi/public-rhai/rhoai/3.0/rocm-ubi9/simple/	registry.redhat.io/rhai/base-image-rocm-rhel9:3.0.0-1763044801	https://catalog.redhat.com/software/containers/rhai/base-image-rocm-rhel9/690377f9e1522d6afa972cc9

Notes:

- NVIDIA CUDA, AMD GPU, and AMD ROCm RPM repositories are configured, but disabled.
- The images listed in Table 2.1 have RHEL RPM repositories enabled. A RHEL RPM is a package file used for the Red Hat Package Manager system on Red Hat Enterprise Linux (RHEL). An RPM file contains all the necessary components for an application, such as executable files, configuration files, and documentation. It simplifies the process of distributing, installing, and managing software by bundling everything into a single, standalone file. You can install additional RPMs, but you need a Red Hat subscription and you must run your container image in root mode (for example, **podman run --user 0**).

For more information about Red Hat Package Manager, see [Introduction to RPM](#).

2.2. MIRROR THE PYTHON INDEX FOR YOUR DISCONNECTED ENVIRONMENT

If you are using a disconnected environment, use the following code example to access the Red Hat Python index content and copy it locally. You can then upload the packages into your own internal hosting service:

```
#!/bin/bash -x

URL=https://console.redhat.com/api/pypi/public-rhai/rhoai/3.0/cuda-ubi9-test/simple/

wget \
--verbose \
--mirror \
--continue \
--no-host-directories \
--cut-dirs=4 \
$URL
```

2.3. INSTALL PACKAGES

To ensure reliable and secure access to supported libraries, start your model customization workflow by creating a workbench image that is based on a Red Hat base image that is configured to use the Red Hat Python index. These base images are listed in Table 2.1.

For guidance on custom workbenches, see [Creating a custom workbench image from your own image](#).

When you use one of the images listed in Table 2.1 as a base image, both **pip** and **uv** commands are pre-configured to use the Red Hat Python index and system trust store for HTTPS.

When you run a **pip install** command, it installs the package version referenced in the Red Hat Python index, ensuring that you are installing a version of the library that is secure and reliably accessible.

For example, use the following commands to install the model customization libraries:

- Install the data processing library:

```
pip install docling
```

- Install the synthetic data generation library:

```
pip install sdg-hub
```

- Install the model training library:

```
pip install training-hub
```

Install the model training library with CUDA support:

```
pip install training-hub[cuda]
```

Note: For additional options and details for installing the model training library, see [Training Hub installation guidelines](#).

2.4. IMPORT EXAMPLE NOTEBOOKS

To get started with customizing your models, you can run provided example notebooks and scripts. Table 2.2 lists the Git repositories that provide example notebooks for each model customization component.

For a comprehensive tutorial that demonstrates an AI/ML workflow, see the [Knowledge Tuning example](#) on the Red Hat AI examples site.

The Knowledge Tuning tutorial is a curated collection of Jupyter notebooks that includes examples of using Docling to process data, training-hub to fine-tune a model on that data, and KServe to deploy the final model for a Question and Answer application.

Table 2.2. Model customization example notebooks

Model customization component	Git clone example repository	Branch	Directory
Data processing using docling	https://github.com/opendatahub-io/data-processing.git	stable-3.0	notebooks/
Synthetic data generation	https://github.com/Red-Hat-AI-Innovation-Team/sdg_hub.git	main	examples
Training	https://github.com/Red-Hat-AI-Innovation-Team/training_hub.git	main	examples
End-to-end example for model customization with these components	https://github.com/red-hat-data-services/red-hat-ai-examples.git	main	knowledge-tuning

2.4.1. Clone an example Git repository

Follow these steps to clone a Git repository from the JupyterLab environment provided with your OpenShift AI workbench.

Prerequisites

- You have the https URL and branch for one of the example Git repositories listed in Table 2.2.

Procedure

- From the OpenShift AI dashboard, go to the project where you created a workbench.
- Click the link for your workbench. If prompted, log in and allow JupyterLab to authorize your user.
Your JupyterLab environment window opens.

The file-browser window shows the files and directories that are saved inside your own personal space in OpenShift AI .

3. Bring the content of an example Git repo inside your JupyterLab environment:
 - a. On the toolbar, click the **Git Clone** icon.
 - b. Enter a Git https URL.
 - c. Select the **Include submodules** option, and then click **Clone**.
4. If you want to use a branch other than **main** (for example, the data processing example repo uses the **stable-3.0** branch), change the branch:
 - a. In the left navigation bar, click the **Git** icon, and then click **Current Branch** to expand the branches and tags selector panel.
 - b. On the **Branches** tab, in the **Filter** field, enter the branch name.
 - c. Select the branch.
The current branch changes to the branch that you selected.

Verification

- In the file browser, double-click the newly-created directory to see the example files.

CHAPTER 3. PREPARE YOUR DATA FOR AI CONSUMPTION

To prepare your data, use Docling to transform unstructured data (such as text documents, images, and audio files) into structured formats that models can consume.

To automate data processing tasks, you can build Kubeflow Pipelines (KFP). For examples of pre-built pipelines for unstructured data processing with Docling, see <https://github.com/opendatahub-io/data-processing>.

3.1. PROCESS DATA BY USING DOCLING

Docling is the Python library that you use to prepare unstructured data (like PDFs and images) for consumption by large language models.

3.2. EXPLORE THE DATA PROCESSING EXAMPLES

To get started with data processing with Docling explore the provided examples.

Prerequisites

- Install the data processing library as described in [Set up your working environment](#).

Procedure

1. To access the data processing examples, clone the [data processing Git repository](#):
 - To clone the <https://github.com/opendatahub-io/data-processing.git> repository from JupyterLab, follow the steps in [Clone an example Git repository](#) and specify the 3.0 branch.
 - To create a local clone of the repository, run the following command:

```
git clone https://github.com/opendatahub-io/data-processing -b stable-3.0
```

2. Go to the [notebooks](#) directory to learn how to use Docling for the following tasks:
[Use cases](#)

- Convert unstructured documents (PDF files) to structured format (Markdown) - with and without vision-language model (VLM)
- Chunk - Split documents into smaller, semantically meaningful pieces
- Information extraction - Use template formats to extract specific data fields from documents like invoices.
- Subset selection - Use this script or notebook to reduce the size of your dataset. The algorithm analyzes an input dataset and reduces it in size, while ensuring data diversity and coverage.

[Tutorials](#) - An example notebook that provides a complete, end-to-end workflow for preparing a dataset of documents for a RAG (Retrieval-Augmented Generation) system.

Additional resources

- Docling community project: <https://docling-project.github.io/docling/>

- GitHub Repository for the Docling project source code: <https://github.com/docling-project/docling>

3.3. AUTOMATE DATA PROCESSING STEPS BY BUILDING AI PIPELINES

With Kubeflow Pipelines (KFP), you can automate complex, multi-step Docling data processing tasks into scalable workflows.

With the KFP Software Development Kit (SDK), you can define custom components and stitch them together into a complete pipeline. The SDK allows you to fully control and automate Docling conversion tasks with specific parameters.

Note: You can build a custom runtime image to ensure that all required Docling dependencies are present for pipeline execution. For information on how to run a Docling pipeline with a custom image see the [Docling Pipeline documentation](#).

3.4. EXPLORE THE KUBEFLOW PIPELINE EXAMPLES

To get started with kubeflow pipelines, explore the provided examples. You can download and modify the example code to quickly create a Docling data processing or model training pipeline.

Prerequisites

- Install the data processing library as described in [Set up your working environment](#).

Procedure

1. To access the [kubeflow pipeline examples](#), run the following command to clone the [data processing Git repository](#):

```
git clone https://github.com/openshift-io/data-processing -b stable-3.0
```

2. Go to the **kubeflow-pipelines** directory which contains the following tested examples for running Docling as a scalable pipeline. For instructions on how to import, configure, and run the examples, see the [README file](#) and the Red Hat AI [Working with AI pipelines](#) guide.
 - Standard Pipeline: For converting standard documents that contain text and structured elements. For more information, see the [Standard Conversion Pipelines documentation](#).
 - VLM (Vision Language Model): For converting highly complex or difficult-to-parse documents, such as those with custom instructions or complex layouts, or to add image descriptors. For more information, see the [VLM Pipelines documentation](#).

CHAPTER 4. GENERATE SYNTHETIC DATA

When you customize a model for your enterprise, you must generate high-quality synthetic data to augment your dataset, improve model robustness, and cover edge cases.

Red Hat provides the Synthetic Data Generation (SDG) Hub, a modular Python framework for building synthetic data generation pipelines by using composable blocks and flows. Each block performs a specific task, such as LLM chat, parse text, evaluate, or transform data. Flows chain blocks together to create complex data generation pipelines that include validation and parameter management. A flow (data generation pipeline) is a YAML specification that defines an instance of a data generation algorithm.

4.1. EXPLORE THE SDG HUB EXAMPLES

To get started with SDG Hub, explore the provided examples.

Prerequisites

- Install the Synthetic Data Generation (SDG) Hub library as described in [Set up your working environment](#).

Procedure

1. To access the SDG Hub examples, clone the [SDG Hub Git repository](#):

- To clone the https://github.com/Red-Hat-AI-Innovation-Team/sdg_hub.git repository from JupyterLab, follow the steps in [Clone an example Git repository](#).
- To create a local clone of the repository, run the following command:

```
git clone https://github.com/Red-Hat-AI-Innovation-Team/sdg_hub
```

2. Go to the [examples](#) directory to view the notebooks and YAML files for these use cases:

- **Knowledge tuning** - Generate data to fine-tune a model on enterprise documents so that the resulting trained model can accurately recall relevant content and facts in response to user queries. This example provides a complete walkthrough of data generation and preparation for training.
- **Text analysis** - Generate data for teaching models to extract meaningful insights from text in structured format. Create custom blocks and extend existing flows for new applications. Each use case directory includes a README file that provides details for each use case – such as instructions, performance notes, and configuration tips.

3. When you run the example notebooks, consider the following information:

- **Data generation time and statistics:** The total time to generate data depends on both the maximum concurrency supported by your endpoint and the complexity of the running flow. Longer flows, such as the flows in the Knowledge Generation notebooks, take more time to complete because they produce a large number of summaries and Q&A pairs, each of which undergoes verification within the pipeline.
- **LLM endpoint requirements:** For running flows in the Knowledge Generation notebooks, Red Hat recommends that you set the following values:
 - Set **NUMBER_OF_SUMMARIES** to a minimum of 10.

- To achieve reasonable data generation times and avoid timeouts, use an endpoint that supports a maximum concurrency of at least 50.
- Extend LiteLLM's request timeout by setting the environment variable **LITELLM_REQUEST_TIMEOUT**.

Additional resources

- SDG community documentation: <https://github.com/instructlab/sdg/tree/main/docs>
- SDG GitHub repository: <https://github.com/instructlab/sdg>

4.2. GUIDED EXAMPLE - BUILD A KFP PIPELINE FOR SDG

You can generate synthetic data for domain-specific model customization by using a Kubeflow Pipeline (KFP) on Red Hat OpenShift AI. The [Domain Customization Data Generation using Kubeflow Pipelines \(KFP\)](#) is a guided example.

Prerequisites

- Install the Synthetic Data Generation (SDG) Hub library as described in [Set up your working environment](#).

Procedure

1. Run the following command to clone the (org-name) AI examples repository that includes the KFP pipeline for knowledge tuning example.

```
git clone https://github.com/red-hat-data-services/red-hat-ai-examples
```

2. Navigate to the **examples/domain_customization_kfp_pipeline** directory.
3. Follow the instructions in the README file to run the example:
 - a. Configure an environment variable (**.env**) file, provide your model endpoint, and store the file as a Kubernetes secret. The KFP pipeline consumes the secret as environment variables.
 - b. Generate the KFP pipeline YAML file.
 - c. Upload the YAML file to OpenShift AI and deploy the pipeline.

Verification

The example pipeline generates three types of document augmentations and four types of QA on top of 3 augmentation and 1 original document. It stores the generated data in the Cloud Object Storage (COS) bucket that is linked through the pipeline server.

CHAPTER 5. TRAIN THE MODEL BY USING YOUR PREPARED DATA

To train the model, you can use the Red Hat Training Hub and the KubeFlow Trainer Operator.

You can simplify and accelerate the process of fine-tuning and customizing a foundation model by using your own data. The Red Hat Training Hub is an algorithm-focused interface for common LLM training, continual learning, and reinforcement learning techniques.

5.1. EXPLORE THE TRAINING HUB EXAMPLES

The Training Hub repository hosts multiple cookbooks for using different LLM algorithms such as supervised fine tuning (SFT) and Orthogonal Subspace Fine Tuning (OSFT)/Continual Learning. OSFT is a training algorithm built by the Red Hat AI Innovation team. With OSFT, you can continually post-train a fine-tuned model to expand its knowledge on new data. You can tinker with the Training Hub cookbooks from a workbench within your OpenShift AI project.

To get started with the Training Hub, explore the provided examples.

Prerequisites

- Install the Training Hub library as described in [Set up your working environment](#).

Procedure

1. To access the Training Hub examples, clone the [Training Hub Git repository](#):
 - To clone the https://github.com/Red-Hat-AI-Innovation-Team/training_hub.git repository from JupyterLab, follow the steps in [Clone an example Git repository](#).
 - To create a local clone of the repository, run the following command:

```
git clone https://github.com/Red-Hat-AI-Innovation-Team/training_hub
```

2. Go to the [examples](#) directory to view the Training Hub notebooks, Python scripts and documentation.
 - For a quick overview and descriptions of the supported algorithms and features, with links to examples and getting started code, see the top-level **README** file.
 - For detailed parameter documentation, see the **docs** directory.
 - For hands-on learning with the interactive notebooks, see the **notebooks** directory.
 - For pre-written, configurable python scripts to run training algorithms with various language models, see the **scripts** directory.

5.2. ESTIMATE MEMORY USAGE

To learn how to estimate the amount of memory you need for running and training a specific model, as well as whether your configured GPUs can support the model, use the memory estimator. The memory estimator currently supports only supervised fine tuning (SFT) and Orthogonal Subspace Fine Tuning (OSFT) algorithms. See the following example files in the [Training Hub Git repository](#):

- For the Memory Estimator API, see the **src/training_hub/profiling/memory_estimator.py** file.

- For an example notebook that uses the API, see [notebooks/memory_estimator_example.ipynb](#) file.

5.3. COMPARE THE PERFORMANCE OF OSFT AND SFT TRAINING ALGORITHMS

You can use the OSFT (Orthogonal Subspace Fine-Tuning) and SFT (Supervised Fine-Tuning) algorithms in the Training Hub.

Use SFT to fine-tune a model on supervised datasets with support for:

- Single-node and multi-node distributed training
- Configurable training parameters, for example, epochs, batch size, and learning rate.
- InstructLab-Training backend integration

Use OSFT to fine-tune a model while controlling how much of its existing behavior to preserve, with support for:

- Single-node and multi-node distributed training
- Configurable training parameters (epochs, batch size, learning rate, etc.)
- RHA Innovation Mini-Trainer backend integration

The [examples/docs](#) directory contains information and examples for how to use each algorithm.

Here is a performance comparison of using OSFT and SFT in the Training Hub:

- **Memory scaling:** OSFT memory scales linearly with the unfreeze rank ratio (URR) which is a hyperparameter for OSFT that is a value between 0 and 1 representing the fraction of the matrix rank that is unfrozen and updated during fine-tuning.
A rough comparison can be expressed as OSFT Memory $\sim 3r$ times SFT Memory where r is the URR unfreeze rank ratio – the fraction of the matrix being fine-tuned. At URR = 1/3, OSFT and SFT have similar memory usage.

In most post-training setups, URR values below 1/3 are sufficient for learning new tasks, making OSFT notably lighter in memory.

- **Training time:** On datasets of equal size, OSFT typically takes about twice as long per phase. However, because OSFT does not require replay buffers from past tasks (unlike SFT), the total training time across multiple phases or tasks is lower with clear benefits as the number of tasks grows. Because OSFT supports continual learning without maintaining or reusing old data, it enables lighter, single-pass end-to-end runs.

5.4. DISTRIBUTE TRAINING JOBS BY USING THE KUBEFLOW TRAINER OPERATOR

If you want to implement distributed training across multiple nodes to meet the needs of your training workloads, you can use the KubeFlow Trainer Operator (KFTO). KFTO abstracts the underlying infrastructure complexity of distributed training and fine-tuning of models. The iterative process of fine-tuning significantly reduces the time and resources required compared to training models from scratch.

Learn more about the KubeFlow Trainer Operator in the following OpenShift AI documentation:

- [Running Training Operator-based distributed training workloads](#) in the *Working with distributed workloads* guide.

5.5. DISTRIBUTE FINE-TUNING WITH TRAINING HUB AND KUBEFLOW TRAINER

The KubeFlow Trainer Operator supports distributed fine-tuning by using Training Hub, abstracting the complexity of distributed training. It seamlessly manages scaling and orchestration for you, allowing you to focus on your domain-specific fine-tuning logic by using the simplified Training Hub APIs.

For a comprehensive tutorial on Fine Tuning with Training Hub leveraging distributed nodes with Kubeflow Trainer, follow these guided examples:

- [Example of Supervised Fine Tuning \(SFT\) with Training Hub and Kubeflow Trainer](#)
- [Example of Orthogonal Subspace Fine-Tuning \(OSFT\) with Training Hub and Kubeflow Trainer](#)

CHAPTER 6. END-TO-END MODEL CUSTOMIZATION WORKFLOW

For a comprehensive tutorial that demonstrates an AI/ML workflow, see the [Knowledge Tuning example](#) on the Red Hat AI examples site.

The Knowledge Tuning tutorial is a curated collection of Jupyter notebooks that includes examples of using Docling to process data, training-hub to fine-tune a model on that data, and KServe to deploy the final model for a Question and Answer application.

CHAPTER 7. SUPPORT PHILOSOPHY: A SECURE PLATFORM

Our primary goal is to provide a secure and reliable platform for serving and customizing models on Red Hat OpenShift AI.

The Python packages for model customization (such as **docling**, **sdg-hub**, and **training-hub**) are key components of this platform.

Our support strategy is focused on the integrity of the platform and the secure delivery of these tools, rather than providing direct, standalone support for the individual Python packages themselves.

What is supported

- **Installation on OpenShift AI:** We fully support the successful installation of these packages from the Red Hat AI Python Index onto a supported Red Hat OpenShift AI environment when you use the provided base images.
- **The Platform:** The underlying Red Hat OpenShift AI platform, including its components and infrastructure, is fully supported according to its own lifecycle policy.

What is not supported

- Issues arising from the use of these packages, for example, to build custom flows or applications.
- Mixing packages outside of the packages provided with the Red Hat AI Python Index base images.

The primary benefit of this strategy is a secure software supply chain. By using the Red Hat AI Python Index, you are guaranteed:

- **Red Hat Builds:** You are using Red Hat builds of Python libraries built and delivered by Red Hat and our partners. These builds ensure provenance because Red Hat pulls, scans, and builds all dependencies for the packages.
- **Trusted Source:** The index provides a trusted, secure, and reliable source for your generative AI workflows, especially critical for disconnected (air-gapped) environments.
- **Platform Integrity:** You can be confident that the tools are tested and intended for use on the Red Hat OpenShift AI platform.

For deeper technical questions or contributions related to the packages themselves, we encourage users to engage with the upstream open-source communities.